Sapienza Università di Roma

Department of Mechanical and Aerospace Engineering

Homework 2 — Powered Descent and Landing via Direct Collocation and Successive Convexification

Dynamics and Control of Launch Vehicles

Professor:
Prof. A. Zavoli

Student:
Niccolò D'Ambrosio

Academic Year 2025/2026

Contents

1 Introduction

This report presents the solution to Homework 2 of the course *Dynamics and Control of Launch Vehicles*: the minimum-fuel powered descent and pinpoint soft landing of a reusable launch vehicle. While Homework 1 tackled an ascent problem via the indirect approach (Pontryagin Maximum Principle plus shooting), this work adopts a *direct transcription*: the optimal control problem is approximated by a finite-dimensional Nonlinear Programming (NLP) problem through trapezoidal collocation, which is then solved by Sequential Quadratic Programming (SQP). The optional Task 2 complements this baseline with a Zero-Order Hold transcription and a Successive Convexification (SCvx) loop whose inner sub-problem is solved both as a generic NLP and as a Second-Order Cone Program.

The powered-descent problem models the final phase of flight of a reusable rocket performing a "divert and soft land" manoeuvre, of the kind flown by the SpaceX Falcon 9 first stage and by Mars precision-landing concepts. The vehicle must reach a designated landing pad with zero residual velocity while respecting thrust bounds, a glide-slope corridor, and altitude constraints, and using minimum propellant.

1.1 Vehicle Model and Equations of Motion

The vehicle is modelled as a 2D point mass moving above flat ground with no aerodynamics. The state vector is

$$\mathbf{x}(t) = \big[x,\ y,\ v_x,\ v_y,\ m\big]^T, \tag{1}$$

and the control is the thrust vector $\mathbf{T}(t) = [T_x,\ T_y]^T$. The equations of motion are

$$\begin{aligned}
\dot{x} &= v_x, && (2)\\
\dot{y} &= v_y, && (3)\\
\dot{v}_x &= \frac{T_x}{m}, && (4)\\
\dot{v}_y &= \frac{T_y}{m} - g, && (5)\\
\dot{m} &= -\frac{\|\mathbf{T}\|}{c}, && (6)
\end{aligned}$$

where $g = 9.81$ m/s^2 is the gravitational acceleration and $c = I_{sp}\, g_0$ is the effective exhaust velocity ($I_{sp} = 225$ s, $g_0 = 9.80665$ m/s^2, hence $c \approx 2206$ m/s). As in Homework 1, the problem is non-dimensionalised before being passed to the solver (see Section 3.1); the dimensional form is retained throughout this section for clarity. Compared to Homework 1, the control is the full thrust vector rather than a scalar direction angle, since the magnitude is now also a degree of freedom subject to a bound.

1.2 Boundary Conditions and Mission Parameters

The vehicle starts at a known state and must reach the origin with zero velocity at the prescribed terminal time:

$$\mathbf{x}(0) = \left[x_0,\ y_0,\ v_{x,0},\ v_{y,0},\ m_0\right]^T, \qquad \mathbf{x}(t_f) = \left[0,\ 0,\ 0,\ 0,\ m_f\right]^T, \tag{7}$$

with the final mass m_f free. The mission data taken from Table 1 of the assignment are summarised in Table 1.

Table 1: Mission data.

Quantity	Symbol	Value	Units
Initial position	(x_0, y_0)	$(1000,\ 3000)$	m
Initial velocity	$(v_{x,0}, v_{y,0})$	$(300,\ -200)$	m/s
Initial mass	m_0	2000	kg
Specific impulse	I_{sp}	225	s
Thrust bounds	$T_{\min},\ T_{\max}$	$0,\ 70$	kN
Glide-slope half-angle	$\theta_{\max}$	60	deg
Flight time (fixed)	t_f	38	s

1.3 Path Constraints

Three path constraints are enforced over $[0, t_f]$:

$$T_{\min} \le \|\mathbf{T}(t)\| \le T_{\max} \qquad \text{(thrust magnitude)}, \tag{8}$$

$$|x(t)| \le \tan(\theta_{\max})\, y(t) \qquad \text{(glide-slope)}, \tag{9}$$

$$y(t) \ge 0 \qquad \text{(no underground flight)}. \tag{10}$$

The glide-slope condition (9) confines the vehicle to a cone of half-angle $\theta_{\max}$ apexed at the landing pad, ensuring a safe approach corridor. It is convex in (x, y) and decomposes into the linear pair

$$+x - \tan(\theta_{\max})\, y \le 0, \qquad -x - \tan(\theta_{\max})\, y \le 0. \tag{11}$$

Both forms are exploited downstream: the linearised pair (11) is what is fed to `fmincon` as a smooth inequality.

1.4 Optimal Control Problem

The minimum-fuel powered-descent problem is the Mayer-form OCP

$$
\begin{aligned}
\min_{\mathbf{T}(\cdot)} \quad & J = -m(t_f) \\
\text{s.t.} \quad & \text{Eqs. (2)–(6),} \\
& \text{boundary conditions of Section 1.2,} \\
& \text{path constraints (8)–(10).}
\end{aligned} \tag{12}
$$

This continuous-time problem is approximated by an NLP using trapezoidal direct collocation, as described in Section 2.

2 Trapezoidal Direct Collocation

Direct transcription methods discretise the continuous optimal control problem (12) over a finite mesh of nodes and replace the continuous-time dynamics with algebraic relations among the node values. The resulting finite-dimensional NLP can be solved by general-purpose nonlinear programming solvers without the need to derive the costate equations and transversality conditions of the indirect approach. This is the methodological complement to the PMP-based formulation used in Homework 1.

2.1 Trapezoidal Quadrature Rule

The continuous-time dynamics are written compactly as

$$\dot{\mathbf{x}}(t) = \mathbf{f}\big(\mathbf{x}(t),\, \mathbf{u}(t)\big), \qquad \mathbf{x} \in \mathbb{R}^{n_x},\ \mathbf{u} \in \mathbb{R}^{n_u}. \tag{13}$$

Discretise the time horizon $[0, t_f]$ into N uniformly spaced nodes $\{t_k\}_{k=1}^{N}$ with spacing $\Delta t = t_f/(N-1)$, and let

$$\mathbf{x}_k = \mathbf{x}(t_k), \qquad \mathbf{u}_k = \mathbf{u}(t_k), \qquad \mathbf{f}_k = \mathbf{f}(\mathbf{x}_k, \mathbf{u}_k). \tag{14}$$

Integrating the state equation over $[t_k, t_{k+1}]$,

$$\mathbf{x}_{k+1} - \mathbf{x}_k = \int_{t_k}^{t_{k+1}} \mathbf{f}\big(\mathbf{x}(\tau), \mathbf{u}(\tau)\big)\, \mathrm{d}\tau. \tag{15}$$

Approximating the integrand by a linear interpolation between the two endpoints and applying the **trapezoidal quadrature rule** yields

$$\mathbf{x}_{k+1} - \mathbf{x}_k \;\approx\; \frac{\Delta t}{2}\big(\mathbf{f}_k + \mathbf{f}_{k+1}\big). \tag{16}$$

The local truncation error of the trapezoidal rule is $O(\Delta t^3)$, giving a globally second-order method. The rule is implicit (since $\mathbf{f}_{k+1}$ depends on $\mathbf{x}_{k+1}$) and A-stable, properties that make it well suited to transcribing dynamical systems with moderately stiff modes.

2.2 Defect Constraints

Promoting (16) from approximation to equality constraint produces the **defect equations** of trapezoidal direct collocation:

$$\boldsymbol{\delta}_k \;:=\; \mathbf{x}_{k+1} - \mathbf{x}_k - \frac{\Delta t}{2}\big(\mathbf{f}_k + \mathbf{f}_{k+1}\big) \;=\; \mathbf{0}, \qquad k = 1,\, \ldots,\, N-1. \tag{17}$$

With n_x states this generates $n_x(N-1)$ scalar nonlinear equality constraints. When all defects are satisfied to machine precision, the discrete trajectory is consistent with the continuous-time dynamics to second order in Δt, and the controls are interpreted as the nodal values of a piecewise-linear input.

For the powered-descent problem ($n_x = 5$, $n_u = 2$) each node contributes $n_x + n_u = 7$ decision variables and the defect block carries $5(N-1)$ rows.

2.3 NLP Formulation

The decision vector stacks states and controls node by node:

$$\mathbf{z} = \left[\mathbf{x}_1^T,\ \mathbf{u}_1^T,\ \mathbf{x}_2^T,\ \mathbf{u}_2^T,\ \ldots,\ \mathbf{x}_N^T,\ \mathbf{u}_N^T\right]^T \in \mathbb{R}^{(n_x+n_u)N}. \tag{18}$$

The transcribed problem is

$$\begin{aligned} \min_{\mathbf{z}} \quad & -m_N \\ \text{s.t.} \quad & \mathbf{c}_{\text{eq}}^{\text{lin}}(\mathbf{z}) = \mathbf{0} && \text{(boundary conditions)}, \\ & \mathbf{c}_{\text{eq}}^{\text{nl}}(\mathbf{z}) = \mathbf{0} && \text{(defects)}, \\ & \mathbf{c}_{\text{ineq}}(\mathbf{z}) \leq \mathbf{0} && \text{(path constraints)}, \\ & \mathbf{z}_{\text{lb}} \leq \mathbf{z} \leq \mathbf{z}_{\text{ub}} && \text{(box bounds)}. \end{aligned} \tag{19}$$

With $N = 50$, the dimensions are: $7N = 350$ decision variables; 9 linear equalities (5 initial + 4 final boundary conditions, m_f being free); $5(N-1) = 245$ nonlinear equalities (defects); $4N = 200$ nonlinear inequalities (lower and upper thrust magnitude, two glide-slope half-spaces); and $7N$ box bounds.

2.4 KKT Optimality Conditions

Introducing multipliers $\boldsymbol{\lambda}^{\text{eq}}$ and $\boldsymbol{\mu} \geq \mathbf{0}$ for the equality and inequality constraints, the Lagrangian of (19) is

$$\mathcal{L}(\mathbf{z}, \boldsymbol{\lambda}^{\text{eq}}, \boldsymbol{\mu}) = J(\mathbf{z}) + (\boldsymbol{\lambda}^{\text{eq}})^T \mathbf{c}_{\text{eq}}(\mathbf{z}) + \boldsymbol{\mu}^T \mathbf{c}_{\text{ineq}}(\mathbf{z}). \tag{20}$$

A point $\mathbf{z}^*$ is a local minimiser if there exist multipliers $(\boldsymbol{\lambda}^{\text{eq},*}, \boldsymbol{\mu}^*)$ such that the **Karush–Kuhn–Tucker** (KKT) conditions hold:

$$\nabla_{\mathbf{z}} \mathcal{L}(\mathbf{z}^*, \boldsymbol{\lambda}^{\text{eq},*}, \boldsymbol{\mu}^*) = \mathbf{0} \qquad \text{(stationarity)}, \tag{21}$$

$$\mathbf{c}_{\text{eq}}(\mathbf{z}^*) = \mathbf{0}, \quad \mathbf{c}_{\text{ineq}}(\mathbf{z}^*) \leq \mathbf{0} \qquad \text{(primal feasibility)}, \tag{22}$$

$$\boldsymbol{\mu}^* \geq \mathbf{0} \qquad \text{(dual feasibility)}, \tag{23}$$

$$\mu_j^* \, c_{\text{ineq},j}(\mathbf{z}^*) = 0, \quad \forall j \qquad \text{(complementary slackness)}. \tag{24}$$

Complementarity expresses that an inequality is either active ($c_j = 0$, with $\mu_j > 0$) or strictly inactive ($c_j < 0$, with $\mu_j = 0$). At the optimum of the powered-descent problem the upper thrust bound $\|\mathbf{T}\| = T_{\max}$ is active on the two burn arcs (about half of the nodes), mirroring the max–coast–max structure of the fuel-optimal control law for problems with linear-in-control dynamics; the corresponding KKT multipliers are positive, identifying the binding constraint that shapes the solution.

2.5 Sequential Quadratic Programming

The NLP (19) is solved with MATLAB's `fmincon` configured with the SQP algorithm. SQP iteratively constructs a quadratic-programming approximation of the KKT system: at each iterate $\mathbf{z}^{(k)}$ the search direction $\mathbf{d}^{(k)}$ is found by solving

$$\begin{aligned} \min_{\mathbf{d}} \quad & \tfrac{1}{2}\mathbf{d}^T\mathbf{H}^{(k)}\mathbf{d} + \nabla J(\mathbf{z}^{(k)})^T\mathbf{d} \\ \text{s.t.} \quad & \nabla\mathbf{c}_{\text{eq}}(\mathbf{z}^{(k)})^T\mathbf{d} + \mathbf{c}_{\text{eq}}(\mathbf{z}^{(k)}) = \mathbf{0}, \\ & \nabla\mathbf{c}_{\text{ineq}}(\mathbf{z}^{(k)})^T\mathbf{d} + \mathbf{c}_{\text{ineq}}(\mathbf{z}^{(k)}) \leq \mathbf{0}, \end{aligned} \tag{25}$$

where $\mathbf{H}^{(k)}$ is a quasi-Newton (BFGS) approximation of the Hessian of the Lagrangian. The next iterate is $\mathbf{z}^{(k+1)} = \mathbf{z}^{(k)} + \alpha^{(k)}\mathbf{d}^{(k)}$, with the step length $\alpha^{(k)}$ obtained from a line search on a merit function that balances objective decrease against constraint violation.

3 Numerical Implementation

This section details the practical setup of the NLP (19) in MATLAB. The full transcription is built and solved by the script `main_task1.m`; `fmincon` is the only solver dependency (no CasADi, no YALMIP).

3.1 Non-Dimensionalisation

The decision-vector entries span four orders of magnitude in SI units (positions $\sim 10^3$ m, velocities $\sim 10^2$ m/s, mass $\sim 10^3$ kg, thrust components $\sim 7 \cdot 10^4$ N). Mixing scales of this size in a single NLP yields a poorly conditioned Jacobian and slow asymptotic convergence under finite-difference gradients. Following the same approach used in Homework 1, the problem is therefore non-dimensionalised before being handed to `fmincon`. The reference scales are listed in Table 2.

Table 2: Non-dimensionalisation reference scales.

Quantity	Symbol	Definition	Numerical value
Length	L_{ref}	y_0	3000 m
Acceleration	a_{ref}	g	9.81 m/s^2
Time	t_{ref}	$\sqrt{L_{ref}/g}$	17.49 s
Velocity	V_{ref}	$\sqrt{g\,L_{ref}}$	171.6 m/s
Mass	m_{ref}	m_0	2000 kg
Thrust	T_{ref}	$m_0\,g$	19 620 N

In non-dimensional variables (denoted by tildes), the dynamics (2)–(6) reduce to

$$\dot{\tilde{x}} = \tilde{v}_x, \qquad \dot{\tilde{y}} = \tilde{v}_y, \tag{26}$$

$$\dot{\tilde{v}}_x = \frac{\tilde{T}_x}{\tilde{m}}, \qquad \dot{\tilde{v}}_y = \frac{\tilde{T}_y}{\tilde{m}} - 1, \tag{27}$$

$$\dot{\tilde{m}} = -V_c\,\|\tilde{\mathbf{T}}\|, \tag{28}$$

where $V_c = V_{ref}/c \approx 0.0778$ is the only residual dimensionless parameter (a local Tsiolkovsky-style ratio of the characteristic flight speed to the effective exhaust velocity). All decision variables now span $O(1)$ magnitudes, with positions in $[-1/3,\ 1]$, velocities in $[-1.17,\ 1.75]$, mass in $[10^{-3},\ 1]$, and thrust components in $[-3.57,\ 3.57]$; the Jacobians of the LTV linearisation (Section 6) become uniformly $O(1)$, sharply improving the conditioning of `fmincon`'s SQP iterations and of the SCvx outer loop. Boundary conditions, glide-slope angle, and the time horizon t_f are all rescaled by their natural units; results are returned to SI before printing and plotting.

3.2 Decision Vector Layout

With $N = 50$ collocation nodes, the decision vector (18) has $7N = 350$ components ordered as

$$\mathbf{z} = \big[\underbrace{x_1, y_1, v_{x,1}, v_{y,1}, m_1, T_{x,1}, T_{y,1}}_{\text{node } 1},\ \dots,\ \underbrace{x_N, y_N, v_{x,N}, v_{y,N}, m_N, T_{x,N}, T_{y,N}}_{\text{node } N}\big]^T. \tag{29}$$

A slice operator `idx(i) = (i-1)*7 + (1:7)` returns the indices of the seven entries belonging to node i, used throughout the helper functions to build sparse constraint matrices and unpack the converged solution.

3.3 Initial Guess

A reasonable initial guess is essential for SQP convergence on this nonconvex NLP. The state variables are linearly interpolated between the boundary states with $\alpha = (i-1)/(N-1)$:

$$x_i^{(0)} = (1-\alpha)\, x_0, \qquad y_i^{(0)} = (1-\alpha)\, y_0, \tag{30}$$

$$v_{x,i}^{(0)} = (1-\alpha)\, v_{x,0}, \qquad v_{y,i}^{(0)} = (1-\alpha)\, v_{y,0}, \tag{31}$$

$$m_i^{(0)} = m_0\big(1 - 0.3\,\alpha\big). \tag{32}$$

The control guess is a constant hover thrust $\mathbf{T}^{(0)} = (0,\, m_0\, g)^T$, which approximately balances gravity at the start and provides a search direction that does not violate the thrust-magnitude bounds.

3.4 Constraints

Linear equality (boundary conditions). The nine scalar conditions are encoded in $\mathbf{A}_{\text{eq}}\mathbf{z} = \mathbf{b}_{\text{eq}}$ as a sparse $9 \times 7N$ matrix: five rows fix the initial state and four rows fix the terminal $(x, y, v_x, v_y) = (0, 0, 0, 0)$. The terminal mass is left free; its strictly-positive lower bound is enforced as a box constraint (see below).

Nonlinear equality (defects). The function `trap_nonlcon` reshapes $\mathbf{z}$ into a $7 \times N$ matrix, evaluates the right-hand side $\mathbf{f}_k$ at every node via `dyn_rhs`, and assembles the $5(N-1) = 245$ defect entries of Eq. (17).

Nonlinear inequality. The same routine returns the $4N$ inequalities:

- Thrust magnitude (lower and upper): $T_{\min} - \|\mathbf{T}_k\| \le 0$ and $\|\mathbf{T}_k\| - T_{\max} \le 0$, both nonsmooth at the origin but smoothly differentiable away from $\mathbf{T} = \mathbf{0}$;
- Glide-slope (linear, decomposed): $+x_k - \tan\theta_{\max}\, y_k \le 0$ and $-x_k - \tan\theta_{\max}\, y_k \le 0$.

Box bounds. Per-node bounds enforce $\tilde{y}_k \geq 0$, $10^{-3} \leq \tilde{m}_k \leq 1$ (equivalently $2 \leq m_k \leq m_0$ kg in SI), and $|\tilde{T}_{x,k}|$, $|\tilde{T}_{y,k}| \leq \tilde{T}_{\max}$. The strictly-positive mass lower bound avoids division-by-zero in the dynamics RHS (4)–(5); the chosen value of 2 kg is well below the propellant inventory at any feasible terminal state.

3.5 Solver Settings

The NLP is solved with `fmincon` using the options listed in Table 3.

Table 3: `fmincon` options.

Option	Value
`Algorithm`	`'sqp'`
`MaxIterations`	1000
`MaxFunctionEvaluations`	10^6
`OptimalityTolerance`	10^{-5}
`ConstraintTolerance`	10^{-6}
`StepTolerance`	10^{-10}

The SQP algorithm was preferred over the interior-point method (`fmincon`'s default) because it tracks the active-set structure of the max–coast–max optimum — with the thrust bound binding on entire arcs — more efficiently than barrier-based interior-point iterations, and it copes well with the equality-dense defect block. Gradients are not provided explicitly, so `fmincon` computes them via finite differences; the typical wall time at $N = 50$ is on the order of one minute per solve on the test laptop.

A practical remark on the convergence tail: on the coast arc the optimal control sits exactly at $\mathbf{T} = \mathbf{0}$, which is the nonsmooth point of $\|\mathbf{T}\|$ in the mass equation (6) and in the thrust-magnitude constraint. Under finite-difference gradients this caps the attainable first-order optimality at the 10^{-3}–10^{-4} level (non-dim) and explains why some runs stop at the iteration limit with feasible, engineering-quality iterates rather than at the optimality tolerance (Sections 4–5). The standard remedies — left to the roadmap — are either analytical gradients with a smoothed norm, or the lossless-convexification slack $\Gamma \geq \|\mathbf{T}\|$ of Açikmeşe–Ploen, which removes the norm from the dynamics altogether.

4 Numerical Results

The NLP (19) is solved at the nominal flight time $t_f = 38$ s with $N = 50$ collocation nodes. `fmincon` runs for roughly a minute of wall time on the test laptop and returns a final mass of $m_f = 1403.20$ kg, equivalently 596.80 kg of propellant consumed (about 30% of the initial mass). The solver terminates at the `MaxIterations` cap of 1000 iterations; the constraint residual is below the tolerance of 10^{-6} and the first-order optimality plateaus at $\sim 10^{-3}$–10^{-4} in non-dim units, indicating that the algorithm has located a high-quality but not fully converged local optimum — the plateau is consistent with the nonsmooth coast arc discussed in Section 3 (see Section 5 for the analogous behaviour across the sensitivity sweep).

The four key plots are presented below. All three sensitivity runs (Section 5) are overlaid on the same axes for reference; the discussion in this section focuses on the nominal $t_f = 38.00$ s curve.

4.1 Trajectory and Glide-Slope Corridor

Figure 1 shows the descent trajectory in the (x, y) plane together with the glide-slope corridor (dashed grey lines bounding the cone of half-angle $\theta_{\max} = 60°$). The vehicle enters the corridor from the upper right with significant horizontal velocity, executes a divert manoeuvre to align with the landing pad, and softly touches down at the origin (black triangle). The trajectory approaches the corridor boundary near the apex of the divert turn, at about one third of the flight, but does not reach it: the glide-slope constraint stays (narrowly) inactive over the whole descent, as quantified in Section 4.3.

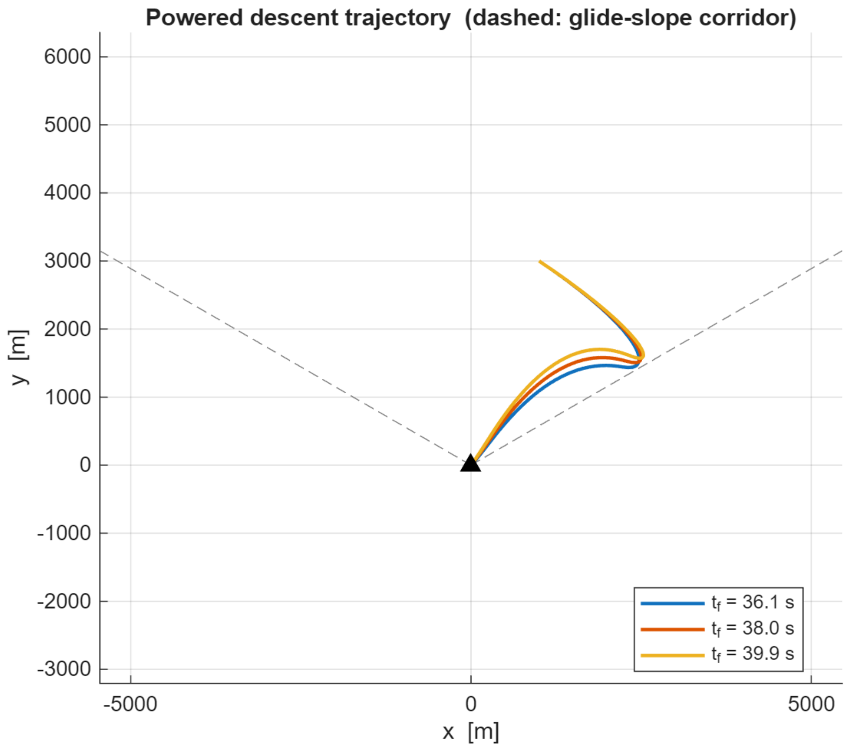


Figure 1: Powered descent trajectory in the (x, y) plane. Dashed grey lines: glide-slope corridor at $\theta_{\max} = 60°$. Black triangle: landing pad at the origin.

4.2 Thrust Magnitude

Figure 2 shows the thrust-magnitude profile $\|\mathbf{T}(t)\|$. The fuel-optimal solution exhibits the expected three-arc **max–coast–max** (bang–off–bang) structure: an initial braking burn at $\|\mathbf{T}\| = T_{\max} = 70$ kN cancels most of the divert velocity, the thrust then drops to essentially zero along a midcourse coast arc, and a second saturated burn performs the terminal braking down to the soft touchdown. For the nominal $t_f = 38$ s the switches occur at $t \approx 14.0$ s and $t \approx 33.1$ s, leaving a coast arc of about 19 s. This is the discrete-time signature of the same Pontryagin-style optimality principle that produced the bilinear-tangent law in Homework 1, applied here to a problem with linear-in-control dynamics and a control-magnitude bound: the optimal thrust magnitude never sits at intermediate values, but switches between the extremes of its admissible set ($T_{\max}$ and the lower bound $T_{\min} = 0$).

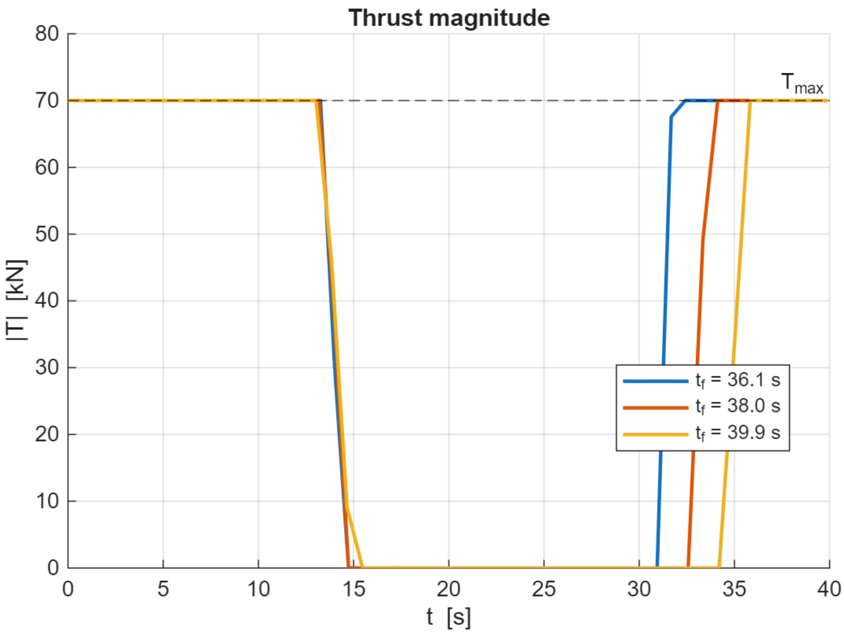
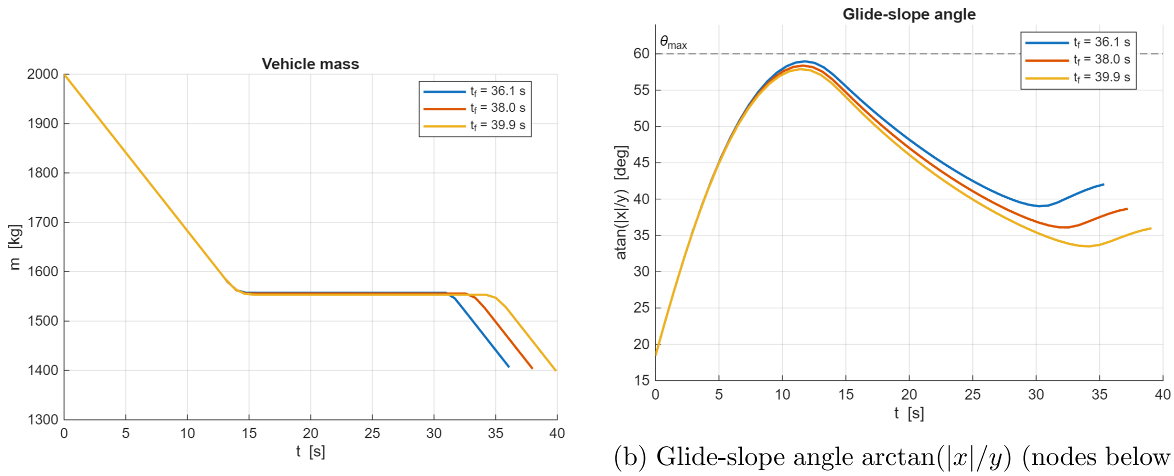


Figure 2: Thrust-magnitude profile. Dashed line: upper bound $T_{\max} = 70$ kN.

4.3 Mass and Glide-Slope Angle

Figure 3a shows the vehicle mass $m(t)$, which decreases at a rate proportional to $\|\mathbf{T}\|$ (Eq. 6): the slope is steep and approximately constant during the two saturated burns and the mass stays constant along the midcourse coast arc, mirroring the max–coast–max thrust profile.

Figure 3b shows the instantaneous glide-slope angle $\theta(t) = \arctan(|x|/y)$, bounded above by $\theta_{\max} = 60°$ (dashed). The angle peaks at $\theta \approx 58.4°$ around $t \approx 12$ s, near the apex of the divert turn, leaving a minimum margin of $\approx 1.6°$: the glide-slope constraint is *never active* at the optimum. The KKT multipliers returned by `fmincon` confirm this quantitatively: the multipliers of the two glide-slope half-space rows are numerically zero at every node, while the upper thrust-magnitude bound carries strictly positive multipliers on the two burn arcs — by complementary slackness, the thrust bound is the only path constraint shaping the solution. Nodes below 1 m altitude are omitted from the plot: as $(x, y) \to (0, 0)$ the ratio $|x|/y$ is numerically indeterminate, and millimetre-level residuals (well within the `ConstraintTolerance` of the linear form (11)) would produce arbitrary spurious angles.



(a) Vehicle mass.

(b) Glide-slope angle $\arctan(|x|/y)$ (nodes below 1 m altitude omitted).

Figure 3: Mass profile and instantaneous glide-slope angle for the nominal solution.

4.4 Grid Convergence

To verify that $N = 50$ nodes are adequate, the nominal problem is re-solved on coarser and finer grids. For each N the converged piecewise-linear control is replayed through the continuous-time dynamics with `ode45` (the same fidelity metric used in Section 6.3) and the maximum position+velocity node error is recorded. The results are summarised in Table 4.

Table 4: Grid-convergence study at the nominal $t_f = 38$ s.

N	m_f [kg]	Max node error [–]	Wall time [s]
25	1402.47	$6.39 \cdot 10^{-3}$	8
50	1403.20	$1.55 \cdot 10^{-3}$	65
100	1402.42	$3.81 \cdot 10^{-4}$	90

The replay error drops by almost exactly a factor of four per mesh halving, a clean experimental confirmation of the $O(\Delta t^2)$ global order of the trapezoidal rule. The final mass, by contrast, fluctuates within ± 0.8 kg ($\approx 0.13\%$ of the propellant) with no systematic trend: the cost landscape is flat enough that the three grids stop at slightly different engineering-quality iterates (the $N = 25$ and $N = 100$ runs stop on the step-size criterion, the $N = 50$ run at the iteration cap). $N = 50$ is retained as the working compromise, with the forward-integration assessment of Section 6.3 as the fidelity cross-check.

5 Sensitivity Analysis

To probe the dependence of fuel consumption on the imposed flight time, the NLP is re-solved at $t_f \in \{0.95, 1.00, 1.05\} \cdot 38$ s, i.e., at 36.10, 38.00, and 39.90 s. Each solve uses the linear-interpolation initial guess of Section 3.3 (no warm-start across runs). The shortest and nominal runs ($t_f = 36.10$ s and 38.00 s) terminate at the `MaxIterations` cap with first-order optimality of order 10^{-3}–10^{-4} in the non-dim cost; the longest run ($t_f = 39.90$ s) satisfies the step-size convergence criterion. The candidate solutions of the two capped runs still respect all constraints to within the `ConstraintTolerance` of 10^{-6}, so they are reported as engineering-quality solutions rather than fully optimised ones.

Table 5 reports the converged final masses and propellant consumption.

Table 5: Sensitivity sweep on flight time.

t_f [s]	m_f [kg]	Fuel consumed [kg]
36.10	1406.33	593.67
38.00	1403.20	596.80
39.90	1398.82	601.18

Fuel use grows with t_f across the swept window. The interpretation is straightforward: the longer the descent, the more time the vehicle spends in the gravity field, and the more thrust must be expended just to counteract gravity (gravity loss). Within the $\pm 5\%$ window around the nominal 38 s the variation is small (≈ 7.5 kg, or about 1.3% of the propellant), indicating that the nominal t_f is close to a flat region of the cost landscape—consistent with the observation that the mission is dominated by horizontal divert and not by hover time.

The variation of the flight time is absorbed almost entirely by the coast arc. Table 6 quantifies this through the switching times, computed as the crossings of the thrust magnitude through $0.5\,T_{\max}$: the first burn is essentially unchanged across the sweep, while the start of the terminal burn shifts to track the imposed t_f.

Table 6: Thrust switching structure across the sensitivity sweep.

t_f [s]	Burn 1 ends [s]	Burn 2 starts [s]	Coast length [s]
36.10	13.91	31.32	17.41
38.00	14.04	33.12	19.09
39.90	14.08	35.04	20.95

The monotonic behaviour suggests that the truly fuel-optimal solution corresponds to the minimum admissible t_f compatible with the thrust-magnitude bound. A free-time formulation, where t_f becomes a decision variable, would seek out this minimum automatically; this is one of the natural extensions identified in Section 7.

The trajectory, thrust, mass, and glide-slope plots presented in Section 4 overlay the three sensitivity runs on common axes, allowing direct visual comparison. The solutions are qualitatively similar: all three exhibit the max–coast–max thrust structure, all three respect the glide-slope corridor, and the trajectories differ only marginally in shape, the extra flight time being spent coasting at altitude before the terminal burn.

6 Task 2: Zero-Order Hold Discretization

The optional Task 2 of the assignment asks for an alternative transcription of the powered-descent problem based on a *Zero-Order Hold* (ZOH) discretisation, validated by forward-integrating the optimised control schedule with a high-fidelity integrator. This section presents **three complementary implementations** and compares them against the trapezoidal-collocation baseline of Task 1:

(a) Nonlinear ZOH with RK4 propagation – a multiple-shooting NLP that holds $\mathbf{u}$ piecewise constant and propagates the *nonlinear* dynamics between grid nodes via a fixed-step RK4 integrator;

(b) LTV-linearised ZOH with SCvx outer loop, solved by `fmincon`/SQP – the literal Appendix A construction: linearise about a reference, compute the discrete-time matrices via the auxiliary ODE, solve the resulting LTV optimisation, and iterate;

(c) LTV-linearised ZOH with SCvx outer loop, solved by `YALMIP`+`ECOS` – same outer loop as variant (b), but the inner sub-problem is modelled in `YALMIP` as a Second-Order Cone Program (SOCP) and solved by the conic solver `ECOS`, exposing the convex structure that variant (b) hides inside a general-purpose NLP.

6.1 ZOH Transcription for LTV Systems (Appendix A)

Appendix A of the assignment derives the ZOH transcription for a continuous-time *linear time-varying* system

$$\dot{\mathbf{x}}(t) = \mathbf{A}(t)\,\mathbf{x}(t) + \mathbf{B}(t)\,\mathbf{u}(t) + \mathbf{c}(t). \tag{33}$$

Discretising time on a uniform grid $\{t_k\}$ with spacing Δt and assuming the control to be piecewise constant on each interval,

$$\mathbf{u}(t) = \mathbf{u}_k \qquad \text{for } t \in [t_k,\, t_{k+1}), \tag{34}$$

the variation-of-constants formula yields the discrete-time recursion

$$\mathbf{x}_{k+1} = \bar{\mathbf{A}}_k\,\mathbf{x}_k + \bar{\mathbf{B}}_k\,\mathbf{u}_k + \bar{\mathbf{c}}_k, \tag{35}$$

where the discrete-time matrices follow from the state-transition matrix $\boldsymbol{\Phi}(t, t_k)$:

$$\bar{\mathbf{A}}_k = \boldsymbol{\Phi}(t_{k+1}, t_k), \quad \bar{\mathbf{B}}_k = \int_{t_k}^{t_{k+1}} \boldsymbol{\Phi}(t_{k+1}, s)\,\mathbf{B}(s)\,\mathrm{d}s, \quad \bar{\mathbf{c}}_k = \int_{t_k}^{t_{k+1}} \boldsymbol{\Phi}(t_{k+1}, s)\,\mathbf{c}(s)\,\mathrm{d}s. \tag{36}$$

These are evaluated numerically by integrating an augmented system over each interval $[t_k,\, t_{k+1}]$. Defining $\hat{\mathbf{B}}(t) = \int_{t_k}^{t} \mathbf{\Phi}(t,s)\mathbf{B}(s)\,\mathrm{d}s$ and $\hat{\mathbf{c}}(t) = \int_{t_k}^{t} \mathbf{\Phi}(t,s)\mathbf{c}(s)\,\mathrm{d}s$, one obtains the linear ODEs

$$\dot{\mathbf{\Phi}} = \mathbf{A}\,\mathbf{\Phi}, \qquad \dot{\hat{\mathbf{B}}} = \mathbf{A}\,\hat{\mathbf{B}} + \mathbf{B}, \qquad \dot{\hat{\mathbf{c}}} = \mathbf{A}\,\hat{\mathbf{c}} + \mathbf{c}, \tag{37}$$

with $\mathbf{\Phi}(t_k, t_k) = \mathbf{I}$ and $\hat{\mathbf{B}}(t_k) = \mathbf{0}$, $\hat{\mathbf{c}}(t_k) = \mathbf{0}$. At the end of each interval $\bar{\mathbf{A}}_k = \mathbf{\Phi}(t_{k+1}, t_k)$, $\bar{\mathbf{B}}_k = \hat{\mathbf{B}}(t_{k+1})$, $\bar{\mathbf{c}}_k = \hat{\mathbf{c}}(t_{k+1})$. This formulation is mathematically equivalent to the $\boldsymbol{\beta}, \boldsymbol{\gamma}$ form of the appendix and avoids the need to invert $\mathbf{\Phi}$ at each step.

6.2 Application to the Nonlinear Powered-Descent Problem

The dynamics (2)–(6) are *nonlinear* in $(\mathbf{x}, \mathbf{u})$ owing to the T_x/m, T_y/m, and $\|\mathbf{T}\|/c$ terms, so Appendix A is not directly applicable. Two strategies are pursued:

Variant (a): nonlinear ZOH with exact RK4 propagation

Keep the nonlinear dynamics intact and propagate the state from $\mathbf{x}_k$ to $\mathbf{x}_{k+1}$ using the constant control $\mathbf{u}_k$:

$$\mathbf{x}_{k+1} = F\big(\mathbf{x}_k,\, \mathbf{u}_k;\, \Delta t\big), \tag{38}$$

where $F(\cdot, \cdot; \Delta t)$ is the ZOH-input flow map of the nonlinear ODE, implemented as a fixed-step RK4 with $n_{\mathrm{sub}} = 2$ substeps per interval. The decision vector and constraint structure are identical to Task 1 except that the trapezoidal defects (17) are replaced by the nonlinear ZOH defects:

$$\boldsymbol{\delta}_k^{\mathrm{ZOH}} \;:=\; \mathbf{x}_{k+1} - F\big(\mathbf{x}_k,\, \mathbf{u}_k;\, \Delta t\big) \;=\; \mathbf{0}, \qquad k = 1, \ldots, N-1, \tag{39}$$

and the terminal control $\mathbf{u}_N$ is pinned to zero (no interval after t_N under the ZOH convention).

Variant (b): linearised LTV ZOH with successive convexification

Linearise the dynamics about a reference trajectory $\big(\mathbf{x}_{\mathrm{ref}}(t), \mathbf{u}_{\mathrm{ref}}(t)\big)$,

$$\mathbf{A}(t) = \left.\frac{\partial \mathbf{f}}{\partial \mathbf{x}}\right|_{\mathrm{ref}}, \quad \mathbf{B}(t) = \left.\frac{\partial \mathbf{f}}{\partial \mathbf{u}}\right|_{\mathrm{ref}}, \quad \mathbf{c}(t) = \mathbf{f}\big(\mathbf{x}_{\mathrm{ref}}, \mathbf{u}_{\mathrm{ref}}\big) - \mathbf{A}\,\mathbf{x}_{\mathrm{ref}} - \mathbf{B}\,\mathbf{u}_{\mathrm{ref}}, \tag{40}$$

yielding the LTV form (33). The discrete matrices $\bar{\mathbf{A}}_k, \bar{\mathbf{B}}_k, \bar{\mathbf{c}}_k$ are computed via Eq. (37) on every interval and used as linear equality constraints (35) in the NLP. With linear dynamics, linear box bounds, linear glide-slope, and convex thrust-magnitude bound, the LTV sub-problem is convex (an SOCP).

The linearisation is only locally accurate, so a **Successive Convexification** (SCvx) outer loop is wrapped around it: at each iteration the LTV sub-problem is solved, the reference is updated to the new solution, and the cycle is repeated until $\|\mathbf{x}^{(j+1)} - \mathbf{x}^{(j)}\|_2 < \tau$. Two implementation choices proved essential for stable convergence on this problem:

- **Warm start from the trapezoidal solution.** Cold-starting SCvx from a linear-interpolation reference with hover thrust drives the LTV NLP into regions where the linearisation predicts mass creation ($\dot{m} > 0$) and other unphysical behaviour, because the offset $\mathbf{c}(t)$ is built from the reference dynamics and only matches the nonlinear $\mathbf{f}(\mathbf{x}, \mathbf{u})$ in a neighbourhood of the linearisation point. Warm-starting from the Task 1 trapezoidal solution places the initial reference close to the optimum and the linearisation is locally faithful from the first iteration.

- **Adaptive box trust region with ratio test.** Even with a good warm start, an unconstrained LTV NLP can drift along directions where the linear model is loose (in particular the mass equation, since $\partial f_5/\partial \mathbf{T}$ is nearly singular when $\|\mathbf{T}\|$ is small): the LTV "predicts" a fuel saving that the nonlinear dynamics do not deliver, and successive iterations oscillate around the true optimum. The implementation therefore wraps the LTV NLP in a hard box trust region with adaptive scale $\rho \in [\rho_{\min}, 1]$, namely $|\tilde{z} - \tilde{z}_{\text{ref}}| \leq \rho\,\boldsymbol{\rho}_0$, with base radii $\boldsymbol{\rho}_0 = (\rho_{xy}, \rho_v, \rho_m, \rho_T) = (0.17,\ 0.6,\ 0.1,\ 1.0)$ in non-dim units (roughly 500 m, 100 m/s, 200 kg, 20 000 N).

  - After each candidate step $\tilde{\mathbf{z}}^{(j+1)}$, the candidate ZOH controls are forward-integrated through the *nonlinear* dynamics with `ode45` to obtain the actual final mass m_f^{act};
  - the LTV-predicted final mass is compared with this through the trust-region ratio $\eta = (m_f^{\text{act}} - m_f^{(j)}) \,/\, (m_f^{\text{pred}} - m_f^{(j)})$;
  - if $\eta < \eta_l = 0.25$ the linear model is unreliable: the step is **rejected**, the reference is left unchanged, and $\rho \leftarrow \rho/2$;
  - if $\eta_l \leq \eta < \eta_h = 0.7$ the step is accepted and ρ is held;
  - if $\eta \geq \eta_h$ the linear model is excellent: the step is accepted and ρ is doubled (capped at the base value).

  This is the canonical SCvx trust-region update and converges to the local optimum without the unphysical mass creation seen with a fixed unsafeguarded box.

The trust region is the cure for *artificial unboundedness.* The companion pathology, *artificial infeasibility*, arises when the linearised constraint set becomes empty even though the nonlinear problem is feasible; the canonical cure adds heavily penalised virtual controls $\boldsymbol{\nu}_k$ to the defect equations. The pathology does surface transiently in this implementation: after the first four rejections shrink the trust region onto the trapezoidal reference (which is not exactly consistent with the ZOH dynamics), the inner solver reports the subproblem infeasible once, in both the `fmincon` and the `ECOS` variants. The loop survives without virtual controls only because the inner solver still returns its least-infeasible iterate, which is then vetted by the nonlinear replay ratio test before being accepted as the new reference. A production implementation — and any cold-started run — should carry the virtual-control safeguard explicitly.

For the nonlinear powered-descent dynamics, the Jacobians evaluate to

$$\mathbf{A} = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & -T_x/m^2 \\ 0 & 0 & 0 & 0 & -T_y/m^2 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}, \qquad \mathbf{B} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 1/m & 0 \\ 0 & 1/m \\ -T_x/(c\|\mathbf{T}\|) & -T_y/(c\|\mathbf{T}\|) \end{bmatrix}. \tag{41}$$

The denominator $\|\mathbf{T}\|$ in $\mathbf{B}$ is regularised with a small additive constant in the implementation to avoid the singularity at $\|\mathbf{T}\| = 0$.

Variant (c): SCvx with a conic inner solver (`YALMIP+ECOS`)

Variant (b) feeds the LTV sub-problem to `fmincon` as a generic NLP, which discards the convex structure of the inner step and pays the cost of a quasi-Newton SQP iteration. Variant (c) keeps the same outer SCvx loop (warm-start from the trapezoidal solution, identical adaptive trust-region update, identical convergence tolerance) but re-models the inner sub-problem as a Second-Order Cone Program. With LTV dynamics $(\bar{\mathbf{A}}_k, \bar{\mathbf{B}}_k, \bar{\mathbf{c}}_k)$ as linear equalities, glide-slope and box bounds as linear inequalities, and the thrust-magnitude bound as the second-order cone constraint $\|\mathbf{u}_k\|_2 \leq T_{\max}$, the inner step is a pure SOCP. The problem is assembled in `YALMIP` via `sdpvar` and solved by `ECOS`, which exploits the conic structure directly. The two outer loops produce nearly indistinguishable trajectories (see Section 6.4) but the YALMIP/ECOS path is markedly cleaner numerically: the inner subproblem retains the cone constraint as a primitive rather than approximating it through SQP linearisations, and the per-step fidelity of the linearisation improves accordingly. The total wall time of the conic variant is approximately 29 s for the full 15-iteration outer loop, about three times faster than the `fmincon`-based variant (b) measured in the same session (Table 8).

6.3 Forward-Integration Validation

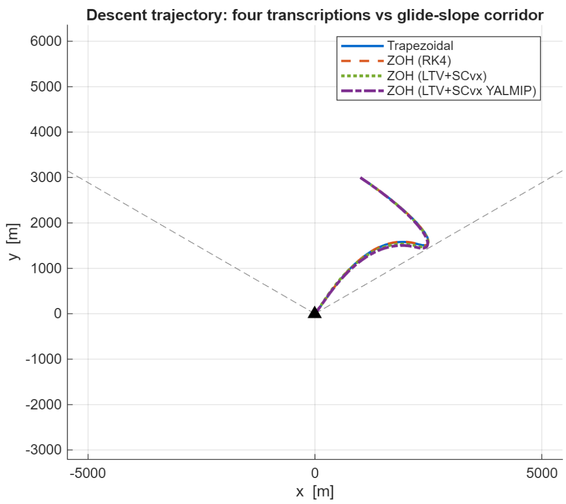
The transcription accuracy of the four approaches (trapezoidal, ZOH–RK4, ZOH–LTV+SCvx with `fmincon`, ZOH–LTV+SCvx with `YALMIP+ECOS`) is assessed by forward-integrating the continuous-time dynamics (2)–(6) with `ode45` (RelTol $= 10^{-10}$, AbsTol $= 10^{-12}$) using the optimised control schedule, and comparing the resulting trajectory against the discretised values at every grid node. The control interpolation matches the transcription assumption: piecewise linear (PWL) for the trapezoidal rule, piecewise constant (PWC) for the three ZOH variants. The node error

$$e_k = \left\| (\mathbf{r}, \mathbf{v})_k^{\mathrm{NLP}} - (\mathbf{r}, \mathbf{v})^{\mathrm{ode45}}(t_k) \right\|_2 \tag{42}$$

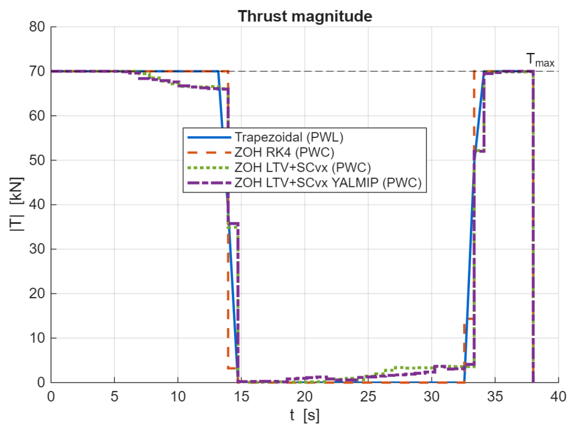
is the norm, in non-dim units, of the position and velocity components; the mass channel is monitored separately through the replayed final mass, which also drives the SCvx ratio test of Section 6.2. This metric quantifies how well each transcription represents the continuous-time problem.

6.4 Results

The four transcriptions converge to qualitatively identical trajectories (Fig. 4a). All four respect the glide-slope corridor and softly land at the origin, with only sub-metric differences in the divert phase. Figure 4b shows the thrust profiles: the trapezoidal solution yields a piecewise-linear thrust whereas the three ZOH variants are naturally piecewise-constant (rendered as staircases). All four exhibit the max–coast–max structure, with $\|\mathbf{T}\|$ saturated at $T_{\max}$ on the two burn arcs and at zero in between.

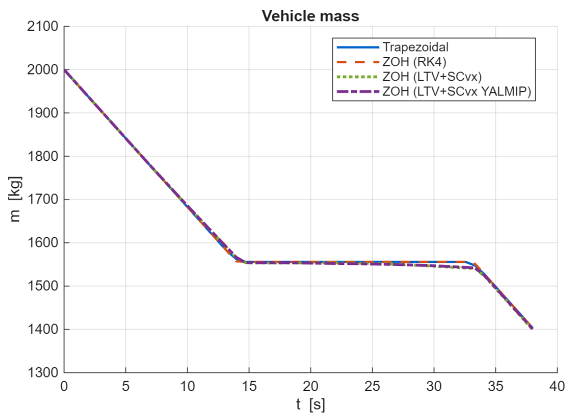


(a) Trajectory in the (x, y) plane.

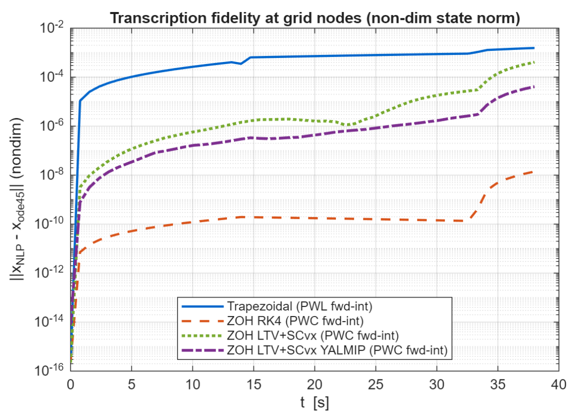


(b) Thrust magnitude: PWL (trapezoidal) vs PWC (ZOH).

Figure 4: Comparison of the four transcriptions at $t_f = 38$ s.



(a) Vehicle mass profile.



(b) Forward-integration error at grid nodes (log scale).

Figure 5: Mass profile and transcription-fidelity check via `ode45` forward integration.

The transcription-fidelity plot (Fig. 5b) is the diagnostic value of Task 2: it measures how far each discretised trajectory drifts from a high-fidelity `ode45` integration of the same controls. A representative numerical comparison is summarised in Table 7.

Table 7: Four-way comparison at $t_f = 38$ s, $N = 50$ nodes.

Transcription	m_f [kg]	Fuel [kg]	Order	Max node error [–]
Trapezoidal	1403.20	596.80	$O(\Delta t^2)$	$1.55 \cdot 10^{-3}$
Nonlinear ZOH (RK4)	1403.37	596.63	$O(\Delta t^4)$	$1.40 \cdot 10^{-8}$
LTV ZOH + SCvx (`fmincon`)	1399.54	600.46	`ode45`-based	$4.08 \cdot 10^{-4}$
LTV ZOH + SCvx (`YALMIP`)	1400.84	599.16	`ode45`-based	$4.09 \cdot 10^{-5}$

The trapezoidal transcription has a global $O(\Delta t^2)$ error and sits at $\sim 10^{-3}$ in non-dim state norm (a few metres of position, fractions of a m/s of velocity). The nonlinear-ZOH+RK4 transcription is $O(\Delta t^4)$ and drops to the `ode45` tolerance floor ($\sim 10^{-8}$). Both LTV+SCvx variants lie in between: they propagate the linearised system with `ode45` on each interval, so the per-step accuracy is high, but the converged solution is constrained by the (small) residual linearisation error within the final trust radius. The conic variant (c) is one order of magnitude cleaner than the `fmincon` variant (b) ($4 \cdot 10^{-5}$ vs $4 \cdot 10^{-4}$): exposing the second-order cone constraint to the inner solver instead of approximating it by SQP linearisations measurably improves the per-step fidelity.

The trapezoidal and nonlinear-ZOH transcriptions agree to within 0.2 kg on m_f, indicating both have located the same local optimum. The two SCvx solutions sit 2–4 kg below this (slightly more fuel burnt) and agree with each other to within 1.3 kg. Two contributing effects are at play:

- Trapezoidal collocation enforces the dynamics only to second order in Δt, so its converged "optimum" is allowed a small dynamical residual ($\sim 10^{-3}$ above) that translates into a marginally optimistic final mass.

- Both SCvx outer loops stop at the 15-iteration cap with $\|\Delta \mathbf{x}\| \approx 10^{-2}$, above the convergence tolerance of 10^{-3}. Every *accepted* step, however, has passed the ratio test against an `ode45` replay of the nonlinear dynamics, so the returned iterate is genuinely realisable; the residual gap simply means the loop settles in a slightly different (but nearby) local optimum.

Both effects are below the engineering accuracy of the model (no aerodynamics, no attitude). Within numerical noise the four transcriptions describe the same physical solution.

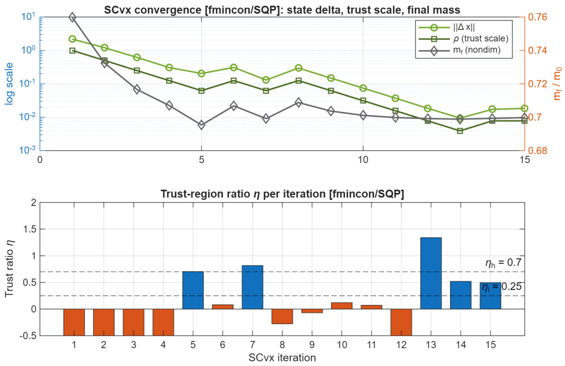
The same `ode45` replay also yields the most operationally meaningful figure of merit: the *touchdown dispersion* obtained by flying each optimised control schedule through the true continuous-time dynamics. Table 8 reports the resulting position and velocity error norms at t_f (target: the origin at rest), the drift of the replayed final mass with respect to the transcription's own m_f, and the wall time of each solve.

Table 8: Replay touchdown dispersion (SI units) and wall time, $t_f = 38$ s, $N = 50$.

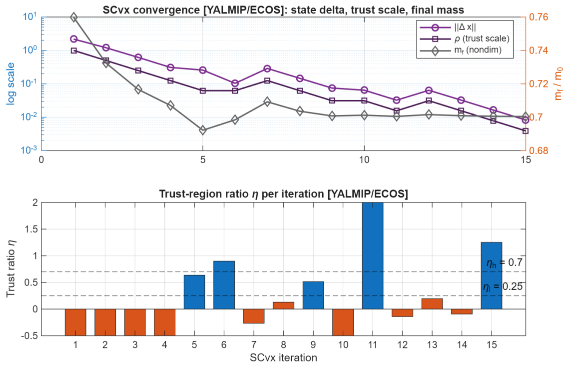
Transcription	Pos. err [m]	Vel. err [m/s]	Δm_f [kg]	Wall [s]
Trapezoidal (PWL)	4.3	0.11	-0.03	90
Nonlinear ZOH (RK4)	$6.1 \cdot 10^{-6}$	$2.4 \cdot 10^{-6}$	$1.6 \cdot 10^{-5}$	100
LTV ZOH + SCvx (`fmincon`)	0.18	0.069	0.45	82
LTV ZOH + SCvx (`YALMIP`)	0.019	0.0069	0.045	29

Flying the trapezoidal PWL controls open-loop lands the vehicle 4.3 m from the pad at 0.11 m/s — entirely adequate for an offline reference trajectory, and exactly the level of dispersion expected from its $\sim 10^{-3}$ transcription error. The ZOH–RK4 schedule replays to micrometre accuracy (its defects *are* the RK4 flow map), while the two SCvx variants land within centimetres to decimetres. All wall times are measured in the same batch session on the same laptop; the conic inner solver makes the SCvx loop about three times faster than every `fmincon`-based solve.

The SCvx convergence traces are reported in Fig. 6. For each variant the top panel overlays the iterate change $\|\mathbf{x}^{(j+1)} - \mathbf{x}^{(j)}\|_2$, the trust-region scale ρ, and the final mass $m_f^{(j)}$; the bottom panel shows the trust-region ratio η against its acceptance band $[\eta_l, \eta_h] = [0.25, 0.7]$ (rejected steps are flagged in orange). Warm-started from the Task 1 trapezoidal solution, both algorithms initially overshoot the linearisation's region of validity and reject the first four candidate steps, shrinking ρ; from then on accepted and rejected steps alternate as the ratio test probes the trust radius, and $\|\Delta\mathbf{x}\|$ settles at the 10^{-2} level by the iteration cap. The final mass m_f stabilises within a few kilograms of the trapezoidal and nonlinear-ZOH values.



(a) Variant (b): SCvx with `fmincon`/SQP inner solver.



(b) Variant (c): SCvx with `YALMIP`+`ECOS` inner solver.

Figure 6: SCvx convergence traces for the two LTV+SCvx variants. Top panels: $\|\mathbf{x}^{(j+1)} - \mathbf{x}^{(j)}\|_2$ (left axis, log), trust scale ρ, and final mass m_f (right axis) per outer iteration. Bottom panels: trust-region ratio η against acceptance band $[\eta_l, \eta_h] = [0.25,\ 0.7]$, with rejected steps highlighted in orange.

The final-mass values returned by the four transcriptions agree to within a few kilograms, confirming that the trapezoidal approximation is accurate enough for engineering purposes despite its lower-order convergence. The ZOH–RK4 variant offers the simplest implementation when only a high-fidelity transcription is needed; the two LTV+SCvx variants expose the convex-sub-problem structure that underpins production guidance algorithms (GFOLD, lossless convexification, *etc.*). The conic variant (c) is the most idiomatic instantiation of that idea – it solves the inner step as an SOCP – and matches variant (b) in trajectory and final mass while delivering a measurably cleaner per-step fidelity.

7 Conclusions

This work transcribed the minimum-fuel powered-descent and pinpoint-landing optimal control problem into a finite-dimensional NLP via trapezoidal direct collocation, and solved it with `fmincon`/SQP. The main findings are as follows.

Direct vs. indirect transcription. Compared to the indirect-shooting approach of Homework 1, direct collocation requires no analytical derivation of costate equations or transversality conditions, accepts path constraints (glide-slope, thrust bounds) natively as inequality constraints, and is robust to a coarse initial guess. The cost is a higher-dimensional NLP (350 variables here) and the loss of the structured analytical insight that the bilinear-tangent law provided in the indirect formulation.

Max–coast–max thrust structure. The fuel-optimal solution exhibits the classical three-arc bang–off–bang behaviour expected for problems with linear-in-control dynamics and a control-magnitude bound: a saturated braking burn at $T_{\max}$, a midcourse coast arc at $T_{\min} = 0$, and a second saturated burn down to touchdown. This is the direct-transcription analogue of the maximum-principle prediction: the optimal thrust magnitude never dwells at intermediate values.

Sensitivity to flight time. A $\pm 5\%$ sweep around the nominal $t_f = 38$ s revealed monotonic growth of fuel consumption with flight time (a longer horizon stretches the coast arc and increases the cumulative gravity loss), with a $\approx 1.3\%$ propellant variation across the window. The lower bound on t_f is set implicitly by the thrust-magnitude constraint.

Active constraints at the optimum. The only path inequality active at the optimum is the upper thrust bound $\|\mathbf{T}\| = T_{\max}$, binding on the two burn arcs with strictly positive KKT multipliers. The glide-slope cone, by contrast, stays narrowly inactive: the trajectory approaches the corridor boundary near the apex of the divert turn with a minimum margin of 1°–2° across the sensitivity sweep, and the corresponding multipliers are numerically zero — complementary slackness thus identifies the thrust bound as the single binding constraint shaping the trajectory.

ZOH transcription (Task 2). The optional ZOH discretisation of Section 6 was implemented in three complementary forms—a nonlinear ZOH with RK4 propagation, the literal Appendix A construction (LTV-linearised + SCvx) solved by `fmincon`/SQP, and the same SCvx outer loop with the inner sub-problem cast as a Second-Order Cone Program and solved by `YALMIP`+`ECOS`. All three confirm the trapezoidal baseline at the engineering level: final masses agree within a few kilograms ($\lesssim 0.7\%$ of the propellant). The forward-integration test exposes the lower convergence order of the trapezoidal rule, with the ZOH–RK4 variant dropping the per-node fidelity error by five orders of magnitude and sitting close to the `ode45` tolerance floor. Warm-started from the trapezoidal solution and stabilised by an adaptive trust-region ratio test, both SCvx variants settle on the same nearby local optimum. The conic variant exposes the SOCP structure of the inner step directly and delivers a per-step fidelity that is an order of magnitude tighter than the `fmincon`/SQP path, demonstrating the convex-sub-problem framework underlying production guidance algorithms (GFOLD, lossless convexification).

Non-dimensionalisation. Following the same approach used in Homework 1, the OCP is solved in non-dimensional variables (Section 3.1), with $L_{ref} = y_0$, $a_{ref} = g$ and the single residual parameter $V_c = V_{ref}/c \approx 0.078$. This shrinks the dynamic range of the decision vector from four orders of magnitude to one and is essential for the LTV linearisation to be well-conditioned: the Jacobians become uniformly $O(1)$ and `fmincon`/SQP makes appreciable progress per iteration.

Limitations and roadmap. The model neglects aerodynamic drag, vehicle attitude dynamics, and any throttling rate limits, and the flight time is fixed. Natural extensions remaining beyond the scope of this report are:

- A **free-time variant** that lifts the fixed-t_f assumption and allows the solver to find the absolute fuel-optimal flight time subject to the active thrust constraint, e.g. as in lossless-convexification formulations.

- **Lossless convexification** of a non-zero lower thrust bound $T_{\min} > 0$, which is non-convex in ($\mathbf{T}$): with the conic SCvx infrastructure already in place (variant (c)), the next step is to introduce the slack-variable trick of Açikmeşe and Ploen to recover a convex SOCP in the new state, and verify that the original constraint is recovered at the optimum.

- The **virtual-control safeguard** against artificial infeasibility in the SCvx inner problem, which surfaced transiently in Section 6: the penalised buffer $\boldsymbol{\nu}_k$ would make the outer loop robust to cold starts.

- Inclusion of **aerodynamic drag** and **attitude dynamics**, moving from a 3-DoF point mass to a 6-DoF rigid body with angle-of-attack and angle-of-sideslip dependence.

References

[1] J. T. Betts. *Practical Methods for Optimal Control and Estimation Using Nonlinear Programming.* SIAM, Philadelphia, 2nd edition, 2010.

[2] C. R. Hargraves and S. W. Paris. Direct trajectory optimization using nonlinear programming and collocation. *Journal of Guidance, Control, and Dynamics*, 10(4):338–342, 1987.

[3] J. Nocedal and S. J. Wright. *Numerical Optimization.* Springer, New York, 2nd edition, 2006.

[4] B. Açikmeşe and S. R. Ploen. Convex programming approach to powered descent guidance for mars landing. *Journal of Guidance, Control, and Dynamics*, 30(5):1353–1366, 2007.

[5] L. Blackmore, B. Açıkmeşe, and D. P. Scharf. Minimum-landing-error powered-descent guidance for Mars landing using convex optimization. *Journal of Guidance, Control, and Dynamics*, 33(4):1161–1171, 2010.

[6] A. E. Bryson and Y.-C. Ho. *Applied Optimal Control: Optimization, Estimation and Control.* Taylor & Francis, New York, revised edition, 1975.

[7] Y. Mao, M. Szmuk, and B. Açıkmeşe. Successive convexification of non-convex optimal control problems and its convergence properties. In *Proceedings of the 55th IEEE Conference on Decision and Control (CDC)*, pages 3636–3641, 2016.

[8] D. Malyuta, T. P. Reynolds, M. Szmuk, T. Lew, R. Bonalli, M. Pavone, and B. Açıkmeşe. Convex optimization for trajectory generation: A tutorial on generating dynamically feasible trajectories reliably and efficiently. *IEEE Control Systems Magazine*, 42(5):40–113, 2022.